

CSCI 6371 — Robotics Programming
Lecture Notes and Homework

Taek Lim
Student ID: 20701313

Fall 2026

Contents

I	Lecture Notes	3
1	Lecture 1: 자율 이동 로봇 개관 (Introduction to Autonomous Mobile Robots)	4
1.1	배경	4
1.2	왜 필요한가	4
1.3	무엇인가	5
1.3.1	지각-사고-행동 순환	5
1.3.2	동작 제어: 바퀴의 종류	6
1.3.3	동작 제어: 로봇 새시와 좌표계	7
1.3.4	지각	8
1.3.5	위치 추정과 인지	9
1.3.6	현황: 오늘날의 로봇	9
1.4	어떻게 적용하는가	9
1.5	앞으로	10
2	Lecture 2: 이동 방식 (Locomotion Concepts)	11
2.1	배경: see-think-act 에서 acting 이 하는 일	11
2.1.1	이동 (locomotion) 이란 무엇인가	11
2.1.2	선행 개념 네 가지	11
2.2	왜 필요한가: 걸을 것인가, 구를 것인가	12
2.3	무엇인가	12
2.3.1	걸을 것인가 구를 것인가: 수송 비용	12
2.3.2	이동 개념의 특성화	13
2.3.3	위치와 방향의 표현	13
2.3.4	다리 로봇	16
2.3.5	걸음새 (gait) 의 수	16
2.3.6	정적 보행과 동적 보행	17
2.3.7	이족보행과 역진자 모형	17
2.3.8	다리 로봇의 역사, 짧게	17
2.3.9	비행 로봇	18
2.3.10	수영 로봇	19
2.4	어떻게 적용하는가	19
2.5	앞으로	19
II	Assignments	20
3	Assignment 1 — Simulator Installation	21
3.1	Environment	21
3.2	Webots	21
3.3	ARGoS	21
3.3.1	The Homebrew tap fails	21

3.3.2	Building from source	22
3.3.3	Verifying the installation	23
3.3.4	Running an example	24
3.3.5	Uninstalling	24
4	Assignment 2 — Rotation Matrices	25
4.1	배경 — 회전 행렬은 무엇이고 왜 필요한가	25
4.1.1	왜 필요한가 — 좌표계가 여러 개다	25
4.1.2	두 가지 해석	25
4.1.3	어떻게 만들어지는가 — 2 차원에서의 유도	25
4.1.4	3 차원으로 확장하기	26
4.1.5	회전 행렬의 성질	26
4.1.6	왜 세 번 나눠서 곱하는가	27
4.2	Elementary rotation matrices	27
4.2.1	Order of composition	27
4.3	Implementation	28
4.4	Question 2 — XYZ rotation	28
4.5	Question 3 — YZX rotation	29
4.6	Results	29
5	Assignment 3 — Low-Pass Filtering of an Image	31
5.1	The problem	31
5.2	Implementation	32
5.3	Results	34

Part I

Lecture Notes

Chapter 1

Lecture 1: 자율 이동 로봇 개관 (Introduction to Autonomous Mobile Robots)

In brief.

Why 이동 로봇 (mobile robot) 이 스스로 움직이려면 “나는 어디 있는가”, “나는 어디로 가야 하는가”, “거기까지 어떻게 가는가” 라는 세 질문에 답해야 한다. 이 장은 그 세 질문을 푸는 표준 틀인 지각-사고-행동 순환 (see-think-act cycle) 을 소개한다.

When 로봇이 바퀴, 다리, 프로펠러 등 어떤 방식으로 움직이든, 그리고 실내 청소 로봇이든 자율주행차든 상관없이 적용되는 일반 틀이다.

Where 이 장은 개관 (overview) 이며 수식은 좌표계 변환 (rotation matrix) 하나뿐이다. 지각·위치 추정·경로 계획 각각의 상세 알고리즘은 이 장의 범위 밖이고, 다음 장들에서 하나씩 다룬다.

How 로봇을 강체 (rigid body) 로 모델링하고, 전역 좌표계 (global frame) 와 로봇 좌표계 (local frame) 사이를 회전 행렬 (rotation matrix) $R(\theta)$ 로 잇는다. 이 계산이 이후 모든 장에서 반복해서 쓰인다.

1.1 배경

이 장은 CSCI 6371 강의의 1 강 슬라이드 (notes/Lecture1.pdf, 총 33 장) 를 옮긴 것이다. 슬라이드 표지에는 이 자료가 Roland Siegwart, Margarita Chli, Nick Lawrance 의 자료 제공 (courtesy) 이라고 명시되어 있을 뿐, 특정 교재의 서지 정보 (저자·출판사·판) 는 적혀 있지 않다. 슬라이드 중간에는 “Ref. to Ch. 2”, “Ref. to Ch. 3” 라는 표시가 여러 번 나오는데, 이 역시 어떤 책의 몇 장인지 이름을 밝히지 않는다. 그래서 이 장에서는 그 표시를 강의가 참고하는 외부 교재의 장 번호로만 언급하고, 책 제목을 지어내지 않는다.

자율 이동 로봇 (autonomous mobile robot) 이란 사람이 매 순간 조종하지 않아도, 스스로 주변을 파악하고 목적지까지 움직이는 로봇을 말한다. 청소 로봇이나 자율주행차가 그 예다. “자율 (autonomous)” 이라는 말은 사람의 개입 없이 스스로 판단한다는 뜻이고, “이동 (mobile)” 은 팔처럼 제 자리에 고정된 로봇과 구분해 몸체 전체가 움직인다는 뜻이다.

1.2 왜 필요한가

로봇이 방을 청소하러 움직이는 상황을 생각해 보자. 목적지가 방 반대쪽이라고 명령만 내리면 로봇은 움직일 수 있을까. 움직이지 못한다. 로봇은 먼저 *지금 자기가 방 어디에 있는지*를 모르면 어느 방향으로 몇 걸음을 가야 할지 계산할 수 없다. 그래서 이동 로봇공학의 모든 문제는 결국 다음 세 질문으로 좁혀진다.

- 나는 어디 있는가 (Where am I?)
- 나는 어디로 가는가 (Where am I going?)
- 거기까지 어떻게 가는가 (How do I get there?)

첫 질문에 답하려면 로봇은 주변 환경의 모형 (model) 을 갖고 있어야 하고 (미리 주어지거나 스스로 만들거나), 센서로 환경을 지각·분석해서 그 모형 안에서 자기 위치를 찾아내야 한다. 둘째와 셋째 질문에 답하려면 그 위치 정보를 바탕으로 움직임을 계획하고 실행해야 한다. 이 세 질문에 대한 답을 하나의 반복 절차로 묶은 것이 다음 절의 지각-사고-행동 순환이다.

1.3 무엇인가

1.3.1 지각-사고-행동 순환

고치는 문제. 세 질문 (절 1.2) 을 한 번만 풀고 끝내면 안 된다. 로봇이 움직이는 동안 환경도 로봇의 위치도 계속 바뀌기 때문에, 센서로 읽고 계산하고 움직이는 절차를 끊임없이 반복해야 한다.

아이디어. 이 반복을 다섯 단계의 순환 고리로 나눈다. 지각 (perception), 위치 추정과 지도 작성 (localization / map building), 인지와 경로 계획 (cognition / path planning), 동작 제어 (motion control), 그리고 그 결과가 실제 환경에 반영되는 단계다. 이 다섯 단계를 통틀어 **지각-사고-행동 순환** (see-think-act cycle) 이라 부른다 (그림 1.1).

- **센싱 (sensing):** 카메라, 라이다 (LiDAR), 초음파 센서 등으로 환경의 원시 데이터 (raw data) 를 얻는다.
- **정보 추출 (information extraction):** 원시 데이터에서 쓸모 있는 특징만 걸러낸다. 잡음을 없애는 필터링 (filtering) · 경계선 찾기 (edge detection), 회전·크기·시점·조명 변화에 잘 버티는 지점을 찾는 키포인트 특징 (keypoint feature, 예: FAST, SURF, SIFT, BRISK), 서로 다른 두 영상에서 같은 키포인트를 짝짓는 키포인트 매칭 (keypoint matching), 사물 단위로 지도를 만드는 객체 수준 의미 지도 작성 (object-level semantic mapping) 이 여기 속한다. 센싱과 정보 추출을 합쳐 **지각 (perception)** 이라 부른다.
- **위치 추정과 지도 작성 (localization / map building):** 추출한 정보와 사전 지식 (knowledge, database) 을 합쳐 “지금 여기가 지도의 어디인지” (position) 를 계산하고, 필요하면 전역 지도 (global map) 를 갱신한다.
- **인지와 경로 계획 (cognition / path planning):** 위치 정보와 임무 명령 (mission commands) 을 받아 목적지까지의 경로 (path) 를 정한다. 지도 전체를 놓고 큰 그림을 짜는 전역 경로 계획 (global path planning, 예: 그래프 탐색) 과, 눈앞의 장애물을 피하는 국소 경로 계획 (local path planning, 예: 국소 충돌 회피) 으로 나뉜다.
- **동작 제어 (motion control):** 정해진 경로를 실제로 따라가도록 구동 명령 (actuator commands) 을 만들어 실행 (path execution) 하고, 그 명령이 모터를 움직여 로봇이 실제로 동작 (acting) 하게 한다.

로봇이 움직이면 환경도 바뀌고, 그 바뀐 환경을 다시 센싱하면서 순환이 이어진다. 그림에서 화살표가 한 바퀴를 도는 것은 이 반복을 뜻한다.

그림 1.1 은 이 장 전체의 지도다. 이후 절에서 다루는 바퀴의 종류, 좌표계 변환은 모두 오른쪽 열 (동작 제어) 의 내부 계산이고, 절 1.3.4 과 절 1.3.5 는 왼쪽 열과 위쪽 행 (인지) 을 조금 더 풀어 쓴 것이다.

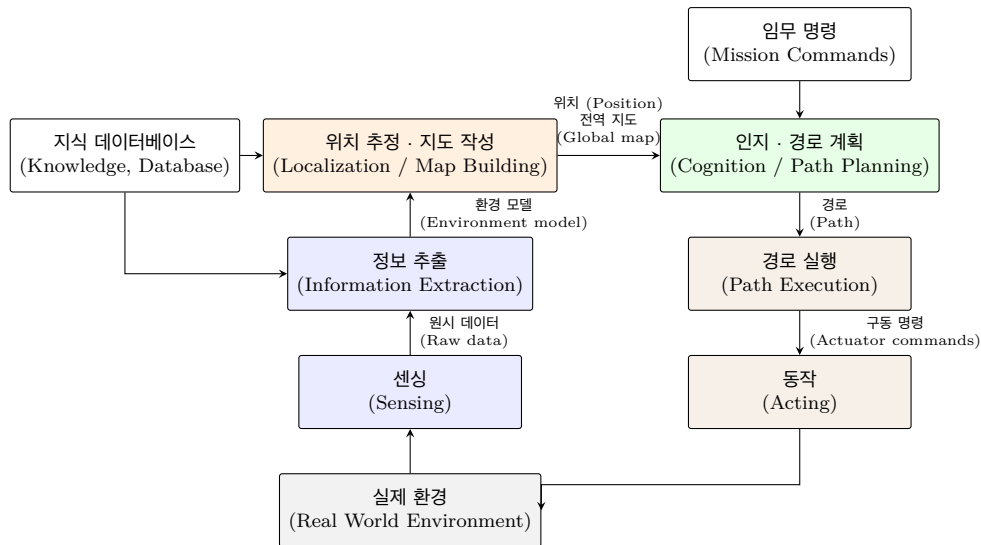


그림 1.1: 지각-사고-행동 순환 (see-think-act cycle). 왼쪽 열은 지각 (perception), 오른쪽 열은 동작 제어 (motion control) 를 이룬다. 다음 장의 구동학 (kinematics) 논의는 이 순환의 동작 제어 단계를 자세히 푼 것이다.

1.3.2 동작 제어: 바퀴의 종류

고치는 문제. 로봇의 동작 제어 (motion control) 를 계산하려면 먼저 바퀴가 지면과 어떻게 접촉하는 지부터 알아야 한다. 바퀴는 접지점에서 두 가지 제약을 받는다. 구르는 방향으로는 미끄러지지 않고 굴러야 한다는 구름 제약 (rolling constraint) 과, 바퀴 축에 수직인 방향 (옆) 으로는 미끄러지지 않는다는 비측면 미끄럼 제약 (no-sliding constraint, lateral) 이다. 어떤 제약이 어느 방향으로 걸리는지는 바퀴의 구조, 즉 바퀴가 몇 개의 축을 중심으로 자유롭게 도는지 (자유도, degrees of freedom, DOF) 에 따라 달라진다.

아이디어. 슬라이드는 로봇에 흔히 쓰이는 바퀴를 네 가지로 분류한다 (표 1.1).

표 1.1: 바퀴의 네 가지 종류와 자유도 (DOF). “어디서 회전하는가” 는 그 바퀴가 자유도를 갖는 회전축의 위치다.

종류 (Type)	자유도 (DOF)	회전이 일어나는 곳	비고
(a) 표준 바퀴 (Standard wheel)	2	(모터로 구동되는) 바퀴 축, 접지점	조향은 되지 않음
(b) 캐스터 바퀴 (Castor wheel)	2	오프셋 조향 관절 (offset steering joint), 접지점	쇼핑카트 앞바퀴 형태
(c) 스웨디시 바퀴 (Swedish wheel)	3	(모터로 구동되는) 바퀴 축, 롤러 (roller), 접지점	롤러가 옆미끄럼을 대신 흡수
(d) 볼/구형 바퀴 (Ball/spherical wheel)	슬라이드에 없음	없음	실현이 기술적으로 어려움 (realization technically difficult)

표 1.1 에서 자유도가 클수록 그 바퀴 하나가 낼 수 있는 움직임의 방향이 다양해진다. 표준 바퀴와 캐

스터 바퀴는 자유도 2 로 앞뒤로만 잘 구르고 옆으로는 미끄러지지 않는 반면, 스웨디시 바퀴는 롤러 축 하나 더 있어 자유도가 3 이 되고, 옆 방향으로도 롤러가 굴러 실질적으로 옆 이동이 가능해진다. 이 차이가 다음 장에서 다룰 옴니휠 (omniwheel) 로봇과 일반 차동구동 (differential drive) 로봇의 움직임 차이로 이어진다 (장 2).

1.3.3 동작 제어: 로봇 새시와 좌표계

In brief.

Why 로봇의 위치와 방향을 숫자로 다루려면 좌표계 (coordinate frame) 가 필요하다. 그런데 로봇 안에서 보는 방향과 방 전체에서 보는 방향은 다르므로, 둘을 잇는 계산이 필요하다.

When 로봇 새시 (chassis) 를 하나의 강체 (rigid body) 로 볼 수 있을 때, 즉 바퀴축·조향 관절의 미세한 자유도는 무시하고 몸체 전체의 위치와 방향만 볼 때 쓴다.

Where 평면 위를 움직이는 바퀴 로봇에 적용된다. 3 차원 공간을 나는 UAV 나 다리로 걷는 로봇은 자유도가 더 많아 이 절의 3 자유도 모델만으로는 부족하다.

How 전역 좌표계 (global frame) 에서 켜 로봇의 자세 (pose) 를 회전 행렬 (rotation matrix) $R(\theta)$ 로 로봇 좌표계 (local frame) 의 값으로 바꾼다.

고치는 문제. 로봇의 바퀴축, 조향 관절, 캐스터 관절 하나하나를 따로 추적하면 계산이 감당할 수 없이 복잡해진다. 그런데 동작 제어에서 정말 궁금한 것은 대개 “로봇 전체가 방 안 어디에, 어느 방향으로 있는가” 이지, 바퀴 하나하나의 자세가 아니다.

아이디어. 그래서 로봇의 몸체 (새시) 를 바퀴처럼 관절이 있는 부분과 분리해서, **하나의 강체**로 취급한다. 평면 위에서 강체 하나의 자세는 숫자 세 개로 완전히 정해진다. 평면 위 위치 두 개 (x, y) 와, 수직축을 기준으로 한 방향 하나 (θ) 다. 즉 로봇 새시는 평면 위에서 **자유도 3** 을 갖는다. 바퀴 축이나 조향 관절이 갖는 추가 자유도는 새시 자체의 자유도에 포함하지 않는다.

예. 서 있는 사람이 방바닥에 원점을 찍고 동서남북으로 축을 그었다고 하자 (전역 좌표계). 로봇이 그 방 안에서 $(x, y) = (2\text{ m}, 1\text{ m})$ 지점에 서서 원래 축에서 30° 돌아서 있다면, 이 로봇의 자세는 세 숫자 $(2, 1, 30^\circ)$ 로 완전히 적힌다. 로봇 안의 왼쪽 바퀴가 정확히 몇 도 돌아있는지는 이 세 숫자에 들어가지 않는다. 그것은 새시가 아니라 바퀴의 자유도이기 때문이다.

정의. 로봇의 자세 (pose) 를 전역 좌표계 $\{X_I, Y_I\}$ 를 기준으로

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

로 적는다. 첨자 I 는 관성계 (inertial frame), 즉 로봇이 움직이든 말든 고정된 방 전체의 좌표계라는 뜻이다. 로봇 몸체에는 그와 별도로 로봇과 함께 움직이는 자기 좌표계 $\{X_R, Y_R\}$ 가 있다. 이때 X_R 은 보통 로봇의 정면, Y_R 은 로봇의 왼쪽을 가리키도록 잡는다 (그림 1.2).

유도. 전역 좌표계의 값 ξ_I 를 로봇 좌표계의 값 ξ_R 로 바꾸는 계산이 필요하다. 이 강의 자료가 쓰는 회전 행렬은

$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1.1)$$

이고, 변환은 $\xi_R = R(\theta) \xi_I$ 다. 행렬을 대입해 한 줄씩 계산하면

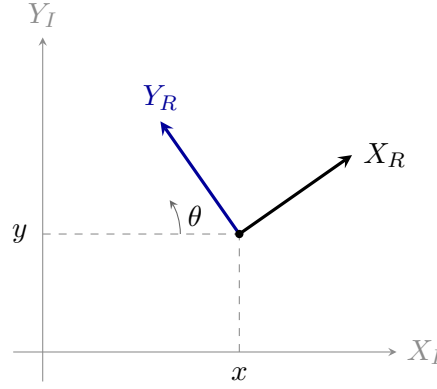


그림 1.2: 전역 좌표계 $\{X_I, Y_I\}$ 와 로봇의 좌표계 $\{X_R, Y_R\}$. 로봇의 자세는 전역 좌표계에서 (x, y, θ) 로 적힌다.

$$\begin{bmatrix} x_R \\ y_R \\ \theta_R \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

$$x_R = x \cos \theta + y \sin \theta$$

$$y_R = -x \sin \theta + y \cos \theta$$

$$\theta_R = \theta$$

행렬 곱을 벌여 쓴다

1 행: R 의 1행과 ξ_I 의 내적

2 행: R 의 2행과 ξ_I 의 내적

3 행: 자세각 자체는 바뀌지 않는다

를 얻는다. 이 값이 바로 식 (1.1) 이 계산하는 것 전부다.

좌표계 변환 규약 (convention) 에 대한 경고. 식 (1.1)의 오른쪽 위 성분은 $-\sin \theta$ 가 아니라 $+\sin \theta$ 다. 이것은 *우연이 아니다*. 식 (1.1)은 벡터를 그대로 두고 *좌표계를 돌려서* 같은 점을 다시 표현하는 **수동적 (passive) 해석**이다 (절 4.1.2 참고). 반면 다음 장에서 쓰는 $R_Z(\theta)$ (식 (4.9))는 좌표계는 그대로 두고 **벡터 자체를 돌리는 능동적 (active) 해석**이고, 부호가 반대다. 사실 이 둘은 서로 전치 (transpose) 관계다. $R(\theta) = R_Z(\theta)^T$ 이고, 회전 행렬은 직교 행렬이므로 $R_Z(\theta)^T = R_Z(\theta)^{-1}$ 다.

구체적인 숫자로 확인해 보자. $\theta = \pi/2$, $\xi_I = (1, 0, \pi/2)^T$ 를 식 (1.1)에 넣으면

$$\xi_R = R\left(\frac{\pi}{2}\right) \xi_I = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ \pi/2 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ \pi/2 \end{bmatrix}$$

로 y 성분이 -1 이 된다. 그런데 같은 $(x, y) = (1, 0)$ 에 장 4의 $R_Z(\pi/2)$ (식 (4.9))를 적용하면

$$R_Z\left(\frac{\pi}{2}\right) \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

로 y 성분이 $+1$ 이 되어 부호가 반대로 나온다. 두 장에서 같은 각도 $\pi/2$ 를 썼는데 결과의 부호가 다른 것은 계산이 틀려서가 아니라, **두 장이 서로 다른 해석의 회전 행렬을 쓰기 때문이다**. 이 장의 $R(\theta)$ 를 다음 장의 $R_Z(\theta)$ 자리에 그대로 옮겨 쓰면 부호가 뒤집힌 답을 얻는다.

1.3.4 지각

지각 (perception) 단계는 절 1.3.1에서 이미 소개했다. 여기서는 그중 정보 추출 (information extraction)에 속하는 키포인트 특징 (keypoint feature)만 조금 더 풀어 쓴다. 카메라 영상 두 장이 조금 다른 각도, 다른 밝기에서 찍혔다고 해도 “같은 지점”을 다시 찾아낼 수 있어야 로봇이 자기 위치를 추정할 수 있다. 그래서 회전, 크기, 시점, 조명이 바뀌어도 잘 안 바뀌는 지점 (키포인트)을 찾는 알고리즘

이 필요하고, FAST, SURF, SIFT, BRISK 같은 이름이 그런 알고리즘들이다. 두 영상에서 뽑은 키포인트끼리 짝을 짓는 작업이 키포인트 매칭 (keypoint matching) 이다.

이보다 한 단계 위에는 객체 수준 의미 지도 작성 (object-level semantic mapping) 이 있다. 각 프레임마다 장면을 기하-의미 분할 (geometric-semantic segmentation) 해서, 이미 알려진 사물과 처음 보는 사물 형태의 요소를 함께 검출하고, 그 결과를 3 차원 볼륨 지도에 누적해 넣는 방식이다. 슬라이드는 이 방식을 구현한 공개 소스 도구로 ASL(취리히 연방공대) 의 voxblox-plusplus 를 든다 (github.com/ethz-asl/voxblox-plusplus), 관련 논문은 Grinvald, Furrer, Novkovic, Chung, Cadena, Siegwart, Nieto, “Volumetric instance-aware semantic mapping and 3D object discovery,” RA-L 2019 이다.

1.3.5 위치 추정과 인지

위치 추정 (localization) 은 “나는 어디 있는가” 라는 첫 질문에 직접 답하는 단계로, 절 1.3.1 의 위치 추정/지도 작성 상자가 하는 일이다. 슬라이드는 이 장에서 위치 추정의 알고리즘 자체는 다루지 않고, 시각-사고-행동 순환에서의 위치만 다시 보여 준다.

인지 (cognition) 는 “나는 어디로 가는가” 와 “거기까지 어떻게 가는가” 에 답한다. 경로 계획은 두 층위로 나뉜다. **전역 경로 계획** (global path planning) 은 지도 전체를 그래프로 보고 그래프 탐색 (graph search) 으로 큰 경로를 찾는다. **국소 경로 계획** (local path planning) 은 눈앞에 갑자기 나타난 장애물처럼 지도에 없던 상황에 대응하는 국소 충돌 회피 (local collision avoidance) 를 맡는다.

1.3.6 현황: 오늘날의 로봇

시각-사고-행동 순환이 실제로 어디까지 와 있는지, 슬라이드가 드는 예 몇 가지를 짧게 정리한다. 이 절은 개관용이므로 최신 수치를 갱신하는 절이 아니라, 슬라이드 시점의 사례를 그대로 옮긴다.

- **3 차원 레이저 센서 (3D laser sensor)**: 자율주행차가 주변 지형을 3 차원으로 스캔하는 데 쓰인다.
- **카메라 기반 차선 추적 (camera lane tracking)**: 카메라 영상만으로 차선, 표지판, 다른 차량을 검출하고 추적한다.
- **시각 정보만으로 비행하는 UAV(vision-only UAV navigation)**: GPS 가 끊긴 환경에서 카메라 한 대와 특징 기반 시각 SLAM(feature-based visual SLAM) 만으로 자율 비행하는 소형 헬리콥터 무리 사례.
- **정밀 농업을 위한 UAV 군집 (UAV swarms for precision agriculture)**: Flourish 프로젝트는 여러 UAV 가 작업을 병렬화해 효율을 높이고, 협력해 정확도를 높이고, 중복 구성으로 강건성을 높이고, 탈중앙화된 알고리즘으로 규모 확장성을 높이는 것을 목표로 둔다.

1.4 어떻게 적용하는가

이 장의 틀은 이 책의 과제 세 개와 각각 맞닿아 있다.

- 장 3 의 시뮬레이터 (ARGoS, Webots) 는 시각-사고-행동 순환 (그림 1.1) 을 실제로 한 바퀴 돌려 보는 자리다. 시뮬레이터 안에서 센싱, 정보 추출, 위치 추정, 경로 계획, 동작 제어가 모두 코드 한 편 안에서 맞물려 돌아가는 것을 직접 확인하게 된다.
- 장 4 의 회전 행렬 계산은 절 1.3.3 에서 다룬 전역 좌표계와 로봇 좌표계 사이의 변환 그 자체다. 다만 두 장의 회전 행렬은 능동적/수동적 해석이 다르므로, 절 1.3.3 의 경고 상자에서 짚은 부호 차이를 그대로 들고 가야 한다.
- 장 5 의 저역통과 필터 (low-pass filter) 는 절 1.3.4 에서 시각 단계의 한 상자로 소개한 필터링 (filtering) 및 경계선 찾기 (edge detection) 를 직접 구현한 것이다. 잡음을 누르고 나서야 키포인트 특징을 뽑을 수 있으므로, 저역통과 필터는 정보 추출의 첫 단계에 해당한다.

1.5 앞으로

슬라이드는 마지막에 아직 풀리지 않은 문제들을 서비스 로봇 (service robot) 의 과제로 제시한다. 로봇이 실제 물리 세계에서 일하려면 다음이 더 필요하다고 든다.

- 불확실하고 일부만 주어진 정보 (uncertain and partially available information) 를 다루는 능력.
- 환경을 보고 느끼고 이해하는 능력, 그리고 촉각으로 상호작용하기 위한 토크/힘 제어 (“소프트 로봇 (soft robot)”).
- 직관적인 사람-기계 인터페이스 (human-machine interface).
- 매일 배우고 적응하는 능력 (평생 학습, lifelong learning).

슬라이드는 이 목록을 풀려면 AI/ML 만으로는 부족하고, 새로운 센싱·구동·로봇 개념 자체가 함께 필요하다고 강조한다.

다리로 걷는 로봇 (legged robot) 은 이 장에서 짧은 영상 링크로만 등장하고 자세히 다루지 않는다. 바퀴 로봇의 구동학 (절 1.3.2, 절 1.3.3) 과 달리 다리 로봇은 접지와 이탈이 번갈아 일어나는 접촉이 있어 지금도 활발한 연구 주제다. 로봇의 이동 방식 (locomotion) 자체는 다음 장인 장 2 에서 이어서 다룬다.

Chapter 2

Lecture 2: 이동 방식 (Locomotion Concepts)

In brief.

Why 로봇이 세상을 움직여 다니는 방법 (이동, locomotion) 을 결정해야 실제로 환경 안에서 일을 할 수 있다. 다리로 걸을지 바퀴로 구를지, 몇 개의 관절과 몇 가지 걸음새 (gait) 를 쓸지는 에너지·제어 복잡도·지형 적응성을 모두 바꾼다.

When 로봇이 지면 (또는 물, 공기) 과 물리적으로 접촉해 움직이는 모든 상황. 바퀴형 이동 로봇, 다리 로봇, 비행 로봇, 수중 로봇이 전부 이 범주에 든다.

Where see-think-act 순환의 마지막 단계인 행동 (acting) 블록, 그중에서도 구동기 명령 (actuator commands) 이 실제 움직임으로 바뀌는 자리에 해당한다.

How 이동 방식을 안정성 (stability), 접촉 특성 (contact characteristics), 환경 종류 (type of environment) 로 특성화하고, 위치와 방향을 벡터와 회전 행렬 (rotation matrix) 로 표현해 정량적으로 다룬다.

2.1 배경: see-think-act 에서 acting 이 하는 일

이 절의 출처는 *Lecture 2: Locomotion Concepts* 슬라이드 1-6 쪽이다.

2.1.1 이동 (locomotion) 이란 무엇인가

이동 (locomotion) 은 로봇 (vehicle) 과 그 환경 사이의 물리적 상호작용이다. 조금 더 풀면, 로봇이 앞으로 나아가려면 지면이나 물, 공기와 힘을 주고받아야 하고, 그 힘을 만들어 내는 것이 구동기 (actuator) 와 그것을 둘러싼 기구 (mechanism) 다. 그래서 이동을 이해한다는 것은 "로봇이 어디로 가는가" 가 아니라 "로봇이 그 힘을 어떻게 만들어 내는가" 를 이해하는 일이다.

이 장은 장 1 에서 그린 see-think-act 순환 (그림 1.1) 의 마지막 조각을 채운다. 그 순환은 감지 (sensing) → 인지 (cognition) → 행동 (acting) 의 흐름이었고, 이동은 그 마지막 화살표, 즉 구동기 명령 (actuator commands) 이 실제 세계의 움직임으로 바뀌는 실행 (execution) 블록에 해당한다. 앞의 두 블록 (감지, 인지) 이 "어디에 있고 어디로 가야 하는가" 를 계산했다면, 이 장은 "그 결정을 물리적으로 어떻게 실현하는가" 를 다룬다.

2.1.2 선행 개념 네 가지

본문에서 바로 쓰이는 용어를 먼저 정리한다.

자유도 (degree of freedom, DOF) 하나의 강체 (rigid body) 나 관절이 독립적으로 움직일 수 있는 방향의 개수. 다리 하나에 DOF 가 많을수록 더 다양한 자세를 만들 수 있지만, 그만큼 제어해야 할 변수도 늘어난다.

무게중심 (center of gravity, COG) 로봇의 질량이 평균적으로 쏠려 있는 지점. 다리 로봇이 넘어지는지 아닌지는 결국 COG 가 지지 다각형 (support polygon, 접촉점들이 만드는 다각형) 안에 있는지로 판가를 난다.

정적 안정성과 동적 안정성 (static vs. dynamic stability) 정적으로 안정하다는 것은 로봇이 그 순간 멈춰도 넘어지지 않는다는 뜻이고, 동적으로만 안정하다는 것은 계속 움직여야만 넘어지지 않는다는 뜻이다. 자전거가 서 있을 때는 넘어지지만 달릴 때는 넘어지지 않는 것과 같은 이치다. 자세 한 대비는 절 2.3.6 에서 다룬다.

걸음새 (gait) 다리 여러 개를 가진 로봇이 다리를 드는 시점 (lift) 과 내려놓는 시점 (touch-down) 을 어떤 순서로 반복하는지를 정한 패턴. 사람이 걷는 방식과 뛰는 방식이 서로 다른 gait 인 것과 같다. 절 2.3.5 에서 정량적으로 다룬다.

2.2 왜 필요한가: 걸을 것인가, 구를 것인가

무슨 문제가 있었나. 자연은 이동 방법을 아주 많이 진화시켰지만, 그중 바퀴를 쓰는 경우는 거의 없다. 반면 사람이 만든 기술 시스템은 대부분 바퀴나 무한궤도 (caterpillar) 를 쓴다. 왜 이런 차이가 생기는가. 바퀴가 회전하려면 몸체와 분리된 채 계속 미끄러지듯 도는 관절이 필요한데, 생물학적 진화는 그런 회전 관절 (rotating joint) 을 만들어 내지 못했다. 반대로 다리로 걷는 방식은 자연에서는 흔하지만, 공학적으로는 구동기 수가 많고 제어가 훨씬 어렵다.

그래서 무엇을 바꿨나. 그래서 로봇을 설계할 때는 ” 걸을 것인가, 구를 것인가 (walking or rolling?) ” 라는 질문에 명시적으로 답해야 한다. 슬라이드가 제시하는 기준은 네 가지다: 구동기 (actuator) 개수, 구조적 복잡도 (structural complexity), 제어 비용 (control expense), 에너지 효율 (energy efficiency), 그리고 굴러야 할 지형 (terrain, 평지·무른 땅·오르막) 이다.

왜 그 변경이 문제를 푸는가. 이 네 기준 중 에너지 효율은 숫자로 비교할 수 있다는 점에서 특히 유용하다. 그 숫자가 수송 비용 (cost of transportation, CoT) 이고, 절 2.3.1 에서 바로 다룬다. 나머지 세 기준 (구동기 수, 구조 복잡도, 제어 비용) 은 다리 로봇 쪽이 불리한 경우가 많다는 점을 절 2.3.4 에서 다시 확인한다.

2.3 무엇인가

2.3.1 걸을 것인가 구를 것인가: 수송 비용

In brief.

Why 걷기와 구르기 중 어느 쪽이 에너지를 덜 쓰는지 지형·속도별로 비교하려면 질량과 이동 거리를 정 규화한 공통 척도가 필요하다.

When 이동 방식을 에너지 관점에서 비교할 때, 또는 로봇의 이동 효율을 생물 (동물) 과 비교할 때 쓴다.

Where 로봇 몸체 전체의 에너지 소비를 다룬다. 개별 관절의 토크나 전류 같은 세부 사항은 다루지 않는다.

How 소비한 에너지 (또는 일률) 를 무게와 이동 거리 (또는 속도) 로 나눠 무차원 값을 만든다.

로봇 두 대를 비교한다고 하자. 하나는 무겁고 하나는 가볍다면, 단순히 ” 에너지를 얼마나 썼는가 ” 만으로는 비교가 불공평하다. 무거운 로봇은 원래 더 많은 에너지가 필요하기 때문이다. 그래서 에너지를 무게로 나누어 표준화한 값이 필요하고, 그것이 수송 비용 (cost of transportation, CoT) 이다.

$$\text{CoT} = \frac{E}{m g d} = \frac{P}{m g v}. \quad (2.1)$$

기호는 등장 순서대로 다음을 뜻한다.

E 이동에 사용한 에너지 (energy), 단위는 줄 (J).

m 로봇의 질량 (mass), 단위는 kg.

g 중력가속도 (gravity), 지구 표면에서 약 9.8 m/s^2 .

d 이동한 거리 (distance), 단위는 m.

P 순간 일률 (power), 단위는 와트 (W). E 를 이동에 걸린 시간으로 나눈 값이다.

v 속도 (speed), 단위는 m/s. d 를 시간으로 나눈 값이므로 오른쪽 등식은 왼쪽 등식의 분자·분모를 모두 시간으로 나눈 것과 같다.

mg 는 로봇의 무게 (weight, 단위 N) 이므로, CoT 는 ”단위 무게를 단위 거리만큼 옮기는 데 쓴 에너지” 다. 분자 (J) 와 분모 ($\text{N} \cdot \text{m} = \text{J}$) 의 단위가 같으므로 CoT 자체는 무차원 (*dimensionless*) 값이다. CoT 가 작을수록 같은 무게를 같은 거리만큼 옮기는 데 에너지를 덜 쓴다는 뜻이니 효율이 좋다.

걷기와 뛰기는 여기에 추가 손실이 있다. 다리로 이동할 때는 몸통, 즉 무게중심 (COG, 절 2.1.2) 이 한 걸음마다 위아래로 오르내린다. 그 오르내림 자체가 일이므로 (중력을 거슬러 COG 를 들어 올렸다가 다시 내려놓는 일), 순수하게 수평으로 미끄러지는 바퀴보다 다리 쪽이 같은 거리를 갈 때 여분의 에너지를 더 쓴다.

예제 – 수치 예시 (저자가 구성함, 슬라이드 값 아님). 질량 $m = 50 \text{ kg}$ 인 다리 로봇이 $d = 100 \text{ m}$ 를 걷는 데 $E = 49,000 \text{ J}$ 을 썼다고 하자. $g = 9.8 \text{ m/s}^2$ 을 쓰면

$$\begin{aligned} mgd &= 50 \times 9.8 \times 100 = 49,000 && \text{분모 (무게} \times \text{거리)} \\ \text{CoT} &= \frac{49,000}{49,000} = 1.0 && \text{무차원} \end{aligned}$$

CoT = 1.0 은 ”몸무게 1 N 을 1 m 옮기는 데 1 J 을 썼다” 는 뜻이다. 같은 로봇이 $v = 1 \text{ m/s}$ 로 걸었다면 100 m 를 가는 데 100 초가 걸리므로

$$P = \frac{E}{t} = \frac{49,000}{100} = 490 \text{ W}, \quad \text{CoT} = \frac{P}{mgv} = \frac{490}{50 \times 9.8 \times 1} = 1.0$$

두 식이 같은 CoT 를 주는 것은 식 (2.1) 의 두 표현이 애초에 시간으로 약분한 관계이기 때문이다. 참고로 실제 이족보행 로봇의 CoT 는 지형과 설계에 따라 1 보다 훨씬 큰 경우가 많고, 바퀴 차량은 대체로 0.1 근처로 훨씬 작다. 이 비교값은 저자가 직관을 위해 임의로 지정한 것이며 특정 로봇의 실측값이 아니다.

2.3.2 이동 개념의 특성화

이동 방식을 설계하거나 비교하려면 ”무엇을 기준으로 좋고 나쁨을 말할 것인가” 가 먼저 정해져야 한다. 슬라이드는 이 기준을 안정성, 접촉 특성, 환경 종류 세 갈래로 나눈다 (표 2.1).

이 세 갈래는 절 2.3.4 의 다리 개수·관절 논의와 절 2.3.9 의 비행 방식 비교에서 되풀이해 등장하는 잣대다.

2.3.3 위치와 방향의 표현

In brief.

Why 로봇의 자세 (pose) 를 계산하려면 위치뿐 아니라 방향 (orientation) 도 숫자로 표현해야 한다. 관절을 하나씩 돌린 뒤 말단이 어디를 향하는지 계산하는 모든 문제 (정기구학 등) 가 이 표현 위에서 돌아간다.

표 2.1: 이동 개념 (locomotion concept) 을 특성화하는 세 갈래 기준.

갈래	세부 항목
안정성 (stability)	접촉점 개수, 무게중심 (COG) 위치, 정적/동적 안정화 (static/dynamic stabilization), 지형의 경사 (inclination of terrain)
접촉 특성 (contact characteristics)	점 접촉인지 면 접촉인지, 접촉각 (angle of contact), 마찰 (friction)
환경 종류 (type of environment)	구조화 정도 (structure), 매질 (medium: 물, 공기, 무른 지면, 단단한 지면)

When 좌표계 하나 안에서 벡터를 회전시키거나, 로봇 몸체 좌표계와 전역 (global) 좌표계 사이를 오갈 때.

Where 이 절은 장 4 의 회전 행렬 배경 (절 4.1) 과 같은 수학을 쓴다. 다만 장 4 가 과제 문제 풀이에 집중했다면, 이 절은 그 행렬이 이동 (locomotion) 문맥에서 등장하는 자리 (로봇의 자세 계산) 를 보인다.

How 위치는 변위 벡터 (displacement vector) 로, 방향은 회전 행렬 (rotation matrix) 로 표현하고, 여러 축을 돌릴 때는 행렬을 곱해 합성한다.

위치와 방향

로봇의 자세 (pose) 는 두 가지로 나뉜다. *위치* (position) 는 한 좌표계의 원점에서 로봇까지의 변위 벡터 (displacement vector) (x, y, z) 로 표현하고, *방향* (orientation) 은 좌표축이 서로 얼마나 돌아가 있는 지를 나타내는 회전 행렬 (rotation matrix) R 로 표현한다.

축 하나를 중심으로 벡터 $[x \ y \ z]$ 를 반시계 방향 (counter-clockwise, CCW) 으로 각도 θ 만큼 돌리는 기본 회전 행렬은 세 개다.

$$R_X(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}, \quad R_Y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}. \quad (2.2)$$

$$R_Z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2.3)$$

이 세 행렬은 장 4 의 식 (4.7) to (4.9) 와 정확히 같은 행렬이다 (같은 유도, 절 4.1.4 참고).

경고: 두 관례를 섞으면 부호가 뒤집힌다. 식 (2.3) 의 $R_Z(\theta)$ 는 오른쪽 위 성분이 $-\sin \theta$ 이고, 벡터를 돌리는 능동적 (active) 관례다 (절 4.1.2 참고). 반면 장 1 의 $R(\theta)$ 는 오른쪽 위 성분이 $+\sin \theta$ 이고, 전역 좌표계를 로봇 좌표계로 옮기는 수동적 (passive) 관례였다. 두 행렬은 서로 전치 (transpose) 관계, 즉 $R_Z(\theta) = R(\theta)^T$ 다. 어느 쪽을 쓰는지 밝히지 않고 두 장의 행렬을 섞어 쓰면 부호가 뒤집힌 답이 나온다. 이 장과 장 4 는 모두 능동적 관례 (R_Z , 식 (2.3)) 를 쓴다.

합성: 순서가 답을 바꾼다

두 축을 연달아 돌리는 경우를 보자. 슬라이드는 Z 축으로 45° 돌린 뒤, "새 X 축" 으로 다시 45° 돌리는 예를 들고, 합성 행렬을 $R_{XZ} = R_X R_Z$ 로 적은 뒤 $V_{\text{new}} = R_{XZ} V$ 로 적용한다.

$$R_{XZ} = R_X(\theta) R_Z(\theta), \quad V_{\text{new}} = R_{XZ} V. \quad (2.4)$$

장 4 의 합성 규칙 (절 4.1.6, 식 (4.10)) 을 따르면, 행렬을 곱했을 때 벡터에 먼저 닿는 쪽 (오른쪽에 있는 행렬) 이 먼저 적용된 회전이다. 식 (2.4) 에서 R_Z 가 오른쪽에 있으므로, 이 식은 **고정된 전역 Z 축으로 먼저 돌리고, 그다음 고정된 전역 X 축으로 돌리는 (외재적, extrinsic) 합성**이다.

경고: 슬라이드의 "새 X 축"이라는 표현. 슬라이드 캡션은 "새 X 축 (the new X axis)" 이라고 적었는데, 이는 보통 몸체와 함께 돌아간 축을 기준으로 다시 돈다는 내재적 (intrinsic) 합성을 뜻하는 말이다. 그런데 실제로 적힌 식 $R_{XZ} = R_X R_Z$ 는 위에서 확인했듯 전역 축 기준의 외재적 합성이다. 만약 캡션 그대로 몸체의 새 X 축을 기준으로 돌리려 했다면 합성 순서는 반대인 $R_Z R_X$ 가 되어야 한다 (절 4.1.6 참고, 외재적 "A 다음 B" 는 내재적 "B 다음 A" 와 같은 행렬이 된다). 이 불일치는 슬라이드 원문 자체에 있는 것으로, 이 장은 슬라이드에 적힌 식 ($R_{XZ} = R_X R_Z$, 전역 축 기준 "Z 다음 X") 을 그대로 채택하고 여기 기록만 남긴다.

핵심은 어느 쪽이 맞고 틀리냐가 아니라, **회전은 교환법칙이 성립하지 않으므로** ($R_A R_B \neq R_B R_A$, 절 4.1.6) 순서를 반드시 함께 명시해야 한다는 점이다. 다음 예로 직접 확인한다.

예제: 순서를 바꾸면 다른 벡터가 나온다. $v_1 = (1, 2, 3)^T$ 에 $\theta = 45^\circ$ ($\cos 45^\circ = \sin 45^\circ = \sqrt{2}/2 \approx 0.7071$) 로 R_X 와 R_Z 를 두 가지 순서로 합성한다.

$$\begin{aligned} R_Z(45^\circ) v_1 &= \left(-\frac{\sqrt{2}}{2}, \frac{3\sqrt{2}}{2}, 3\right) \approx (-0.71, 2.12, 3.00) && Z \text{ 먼저} \\ R_X(45^\circ)(R_Z(45^\circ) v_1) &\approx (-0.71, -0.62, 3.62) && \text{그다음 } X (= R_{XZ} v_1) \\ R_X(45^\circ) v_1 &\approx (1.00, -0.71, 3.54) && X \text{ 먼저} \\ R_Z(45^\circ)(R_X(45^\circ) v_1) &\approx (1.21, 0.21, 3.54) && \text{그다음 } Z (= R_{ZX} v_1) \end{aligned}$$

두 결과 $(-0.71, -0.62, 3.62)$ 와 $(1.21, 0.21, 3.54)$ 는 서로 다른 벡터다. 회전은 길이를 보존하므로 (절 4.1.5) 두 값 모두 $\|v\|^2 \approx 0.5 + 0.39 + 13.12 \approx 14.0$ 과 $1.46 + 0.04 + 12.53 \approx 14.0$ 으로 $\|v_1\|^2 = 1 + 4 + 9 = 14$ 와 일치해 계산 자체는 맞지만, 벡터의 **방향**은 순서에 따라 완전히 다르다.

예제: 벡터를 직접 돌려 본다

예제: X 축으로 90° . $v = (1, 2, 3)^T$ 를 X 축으로 90° CCW 돌린다 ($\cos 90^\circ = 0, \sin 90^\circ = 1$).

$$v' = R_X(90^\circ) v = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \\ 2 \end{bmatrix} \quad \text{슬라이드 답과 일치}$$

예제: Z 축으로 45° . 같은 $v = (1, 2, 3)^T$ 를 Z 축으로 45° CCW 돌린다.

$$v' = R_Z(45^\circ) v = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{3}{\sqrt{2}} \\ 3 \end{bmatrix} \quad \text{슬라이드 답과 일치}$$

예제: 슬라이드의 두 번째 예제, 값을 다시 계산해 확인. 점 $\langle 1, 1, 2 \rangle$ 을 $\theta = 45^\circ$ 로 Z 축 회전한 결과를 물었다.

$$v' = R_Z(45^\circ) \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2}(1) - \frac{\sqrt{2}}{2}(1) \\ \frac{\sqrt{2}}{2}(1) + \frac{\sqrt{2}}{2}(1) \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ \sqrt{2} \\ 2 \end{bmatrix} \approx \begin{bmatrix} 0 \\ 1.41 \\ 2 \end{bmatrix}$$

pdftotext 로 추출한 원문 슬라이드는 이 결과를 "0, 2, 2" 로 보여 준다. 위 계산의 첫째·셋째 성분 (0 과 2) 은 원문과 정확히 일치하고, 둘째 성분만 다르다. 근호 기호 ($\sqrt{}$) 가 PDF 텍스트 추출 과정에서 자주 깨진다는 점 (과제 설명에 명시된 알려진 문제) 을 고려하면, 원문의 "2" 는 $\sqrt{2}$ 에서 근호가 빠진 결과 로 보는 것이 가장 설명이 잘 된다. 이 장은 직접 계산한 (0, $\sqrt{2}$, 2) 를 최종 값으로 채택한다.

2.3.4 다리 로봇

In brief.

Why 바퀴가 지나가지 못하는 지형 (구멍, 계단, 잔해) 에서도 이동해야 할 때 다리가 필요하다.

When 지형이 불규칙하거나 사람이 쓰도록 만들어진 환경 (계단, 좁은 통로) 을 지나야 할 때.

Where 다리 하나당 자유도가 늘어날수록 적응력은 커지지만 에너지·질량·제어 비용도 함께 커진다.

How 다리마다 최소 2 자유도 (들어 올리기 + 휘두르기) 를 주고, 걸음새 (gait) 로 여러 다리의 동작을 맞춘다.

왜 다리가 필요한가. 다리 달린 로봇은 거친 지형에서 적응력과 기동성이 뛰어나다. 바퀴로는 건널 수 없는 구멍이나 틈을 넘을 수 있고, 다리를 팔처럼 써서 물체를 조작할 잠재력도 있다. 하지만 그 대가로 소비 전력과 기계적 복잡도가 크게 늘어난다: 여러 자유도를 동시에 협조시켜 제어해야 하고, 발을 어디에 디딜 지 정하려면 지형의 미세한 굴곡까지 센서로 봐야 한다.

다리 하나에 관절이 몇 개 필요한가. 다리를 앞으로 내밀려면 최소 2 자유도가 필요하다: 들어 올리는 동작 (lift) 과 휘두르는 동작 (swing) 이다. 실제로는 3 자유도를 쓰는 경우가 가장 흔하고, 발목에 4 번째 자유도를 추가하면 걷기와 안정성이 좋아질 수 있다. 사람의 다리는 발가락 구동까지 합치면 7 개가 넘는 주요 자유도를 쓴다. 자유도를 하나 늘릴 때마다 설계와 제어의 복잡도가 늘어나고, 그 대가는 에너지 소비, 제어 난이도, 질량 증가로 돌아온다.

2.3.5 걸음새 (gait) 의 수

고치는 문제. 다리가 k 개인 로봇은 각 다리를 들고 내리는 순서를 자유롭게 고를 수 있다. 이 순서의 가짓수, 즉 서로 다른 걸음새 (distinct event sequence) 의 개수를 세어보면 다리가 늘어날수록 그 수가 어떻게 되는지 감을 잡을 수 있다.

아이디어. 다리 하나는 사건을 두 개 만든다: 드는 사건 (lift) 과 내려놓는 사건 (release, touch-down). 다리가 k 개면 사건은 모두 $2k$ 개다. 그런데 걸음새는 반복되는 주기 (cycle) 이므로 사건 하나를 기준점 ($t=0$) 으로 고정할 수 있고, 남은 $2k - 1$ 개 사건의 순서만 세면 된다. 그 순서의 가짓수가 곧 $(2k - 1)!$ 이다.

$$N = (2k - 1)!. \quad (2.5)$$

$$k = 2 \text{ (이족보행)} : N = (2 \times 2 - 1)! = 3! = 3 \times 2 \times 1 = 6$$

$$k = 6 \text{ (육족보행)} : N = (2 \times 6 - 1)! = 11! = 39,916,800$$

다리가 두 개에서 여섯 개로 늘었을 뿐인데 가짓수는 6 에서 약 4 천만으로 뛴다. factorial(계승) 함수의 성질상 k 가 조금만 커져도 N 이 폭발적으로 커지기 때문이다. 이 폭발은 실제로 쓸 수 있는 걸음새를 사람이 하나하나 나열해서 고를 수 없다는 뜻이고, 그래서 실제 로봇은 이 조합 전체가 아니라 몇 가지 정형화된 걸음새 (삼각보, 파상보 등) 만 골라 쓴다.

표 2.2 는 식 (2.5) 가 준 6 가지 조합을 슬라이드 그대로 나열한 것이다. 여섯 다리 (육족) 로봇의 정적 걸음새 (static gait) 는 항상 세 다리로 이루어진 삼각대 (tripod) 가 지면에 닿아 있도록 짠다. 반대로 네 다리 동물에서 가장 자연스러운 걸음새 (trot, gallop 등) 는 대부분 동적 (dynamic) 걸음새다.

표 2.2: 이족보행 ($k = 2$) 의 여섯 가지 사건 순서. R 은 오른쪽 다리, L 은 왼쪽 다리를 뜻한다.

번호	사건 순서
1	양다리 지지 → R 지지 / L 들림 → 양다리 지지
2	양다리 지지 → R 들림 / L 지지 → 양다리 지지
3	R 만 지지 → 양다리 들림 → 양다리 지지
4	R 지지 / L 들림 → R 들림 / L 지지 → R 지지 / L 들림
5	R 지지 / L 들림 → 양다리 들림 → R 지지 / L 들림
6	R 들림 / L 지지 → 양다리 들림 → R 들림 / L 지지

2.3.6 정적 보행과 동적 보행

절 2.1.2 에서 정의한 정적/동적 안정성을 걸음새에 적용하면 표 2.3 처럼 대비된다.

표 2.3: 정적 보행 (static walking) 과 동적 보행 (dynamic walking) 의 대비.

정적 보행	동적 보행
최소 세 다리가 항상 지면을 지지	지면 접촉 다리가 세 개 미만인 순간이 있음
모든 관절이 그 순간 멈춰도 넘어지지 않음	계속 움직이지 않으면 넘어짐
안전하지만 느리고 비효율적	빠르고 효율적이지만 구동·제어 부담이 큼

2.3.7 이족보행과 역진자 모형

한 걸음은 사실 다각형을 굴리는 동작과 비슷하다. 다각형의 한 변의 길이가 보폭 (step length) 에 해당하고, 보폭이 작아질수록 그 다각형은 원, 즉 바퀴에 가까워진다. 다만 자연은 회전 관절을 발명하지 않았고, 걷는 한 여전히 중력을 거스르는 일을 해야 한다. 그래서 이족보행은 흔히 역진자 (inverted pendulum, 받침점 위에서 거꾸로 서서 흔들리는 진자) 로 모형화한다. 지지하는 다리가 받침점 역할을 하고, 몸통 COG 가 진자의 추처럼 그 위에서 앞뒤로 넘어가듯 움직인다.

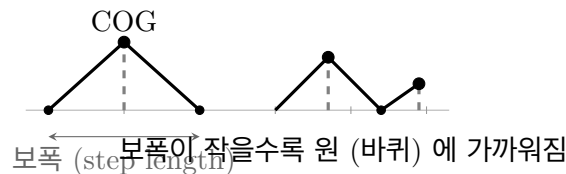


그림 2.1: 이족보행의 역진자 모형. 지지하는 다리가 받침점, 몸통 COG 가 진자의 추 역할을 한다. 보폭이 작을수록 걸음의 궤적은 다각형에서 원에 가까워진다.

탄성 에너지 저장 (elastic energy storage, 힘줄이나 스프링처럼 착지 시 에너지를 저장했다가 다음 걸음에 되돌려주는 방식) 을 활용하면 걸음새가 훨씬 효율적이 된다. 어떤 걸음새가 가장 경제적인지는 고정된 답이 있는 것이 아니라 원하는 속도 (desired speed) 에 따라 달라진다.

2.3.8 다리 로봇의 역사, 짧게

2000 년대 이후 대표적인 이족보행 인간형 로봇은 HONDA ASIMO 다. 2015 년 DARPA Robotics Challenge 에서 여러 팀의 로봇이 문 열기, 계단 오르기 같은 과제에서 넘어지는 모습을 보였는데, 여기서 얻은 교훈은 ”보행은 여전히 어렵다” 는 것이었다. 이후 Boston Dynamics 의 Spot(사족보행) 과 Atlas(이족보행) 가 훨씬 인상적인 기동성을 보여 주었다.

2.3.9 비행 로봇

In brief.

Why 지면이나 물이 아니라 공중에서 이동해야 할 때, 즉 지형 자체를 무시하고 싶을 때 쓴다.

When 넓은 영역을 빠르게 살피거나, 지면 접근이 아예 불가능한 상황.

Where 비행 시간, 기동성, 기계적 복잡도가 방식마다 크게 다르므로 임무 요구사항에 맞춰 골라야 한다.

How 헬리콥터, 고정익, 비행선, 날갯짓 네 방식 중 하나를 고르고, 회전자 속도 차이로 자세 (roll, pitch, yaw) 를 제어한다.

표 2.4 은 무인 항공기 (unmanned aerial vehicle, UAV) 의 네 가지 비행 방식을 비교한다.

표 2.4: 비행 개념 (flight concept) 비교.

방식	비행 시간 및 특징	제약
헬리콥터	20 분 미만	매우 동적이고 기동성이 높음
고정익 (fixed wing) 항공기	수 시간, 연속 비행 가능	비홀로노믹 (non-holonomic) 제약
비행선 (blimp, 공기보다 가벼움)	수 시간 (바람 조건에 의존)	바람에 민감, 탑재 중량 대비 크기가 큼
날갯짓 (flapping wings)	20 분 미만, 활공 (gliding) 가능	비홀로노믹 제약, 기구가 매우 복잡함

여기서 비홀로노믹 (non-holonomic) 이란” 그 순간 갈 수 있는 방향이 원하는 방향 전부가 아니라 일부로 제한된다” 는 뜻이다. 예를 들어 고정익 비행기는 옆으로 미끄러져 이동할 수 없고 앞으로 나아가면서 방향을 서서히 바꿔야 한다. 자동차가 옆으로 바로 평행이동하지 못하는 것과 같은 종류의 제약이다.

쿼드콥터 (quadcopter) 의 자세 제어. 쿼드콥터는 회전자 (rotor) 네 개의 회전 속도 차이만으로 세 가지 회전, 즉 좌우로 기울이는 롤 (roll), 앞뒤로 기울이는 피치 (pitch), 수직축으로 도는 요 (yaw) 를 만든다.

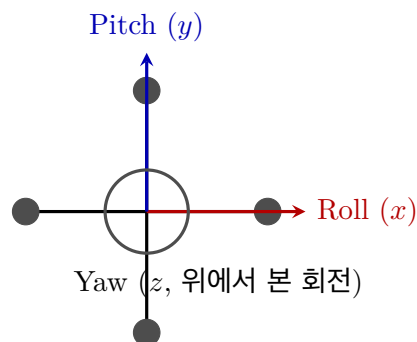


그림 2.2: 쿼드콥터의 세 회전축. Roll 과 Pitch 는 마주보는 회전자 한 쌍의 속도 차이로, Yaw 는 시계 방향/반시계 방향으로 도는 두 쌍의 속도 차이로 만든다.

마주 보는 회전자 두 개 (예: 앞뒤 쌍) 의 속도를 다르게 하면 그 축으로 몸체가 기울며 피치가 생기고, 다른 방향의 쌍을 다르게 하면 롤이 생긴다. 네 회전자는 시계 방향 두 개, 반시계 방향 두 개로 짝지어 회전 반작용 토크를 상쇄하는데, 이 두 쌍의 속도를 서로 다르게 하면 반작용 토크가 상쇄되지 않고 남아 기체가 수직축을 중심으로 요 (yaw) 회전을 한다.

2.3.10 수영 로봇

수중 이동 로봇의 예로는 로봇 물고기 무리 (swarm of robotic fish) 가 있다. 여러 마리가 무리를 지어 헤엄치며 환경을 탐사하는 방식으로, 슬라이드는 이 한 가지 사례만을 제시한다.

2.4 어떻게 적용하는가

이 장의 회전 행렬 대수가 바로 장 4 의 내용이다. 그 과제는 $v_1 = (1, 2, 3)^T$ 에 R_X, R_Y, R_Z 를 XYZ 순서 (절 4.4) 와 YZX 순서 (절 4.5) 로 합성해 서로 다른 결과 v_2, v_3 을 얻는다 (표 4.1). 이때 쓰는 R_X, R_Y, R_Z 는 바로 이 장의 식 (2.2) and (2.3) 와 같은 능동적 (active) 관례다.

아래 코드는 이 장의 R_Z 를 만들어 전역 좌표계와 로봇 좌표계 사이에서 점 하나를 옮긴다. 전치를 취하면 장 1 의 관례로 바뀐다는 점을 주석으로 남겼다.

```
1 import numpy as np
2
3 def rz(theta):
4     c, s = np.cos(theta), np.sin(theta)
5     return np.array([[c, -s, 0],
6                     [s,  c, 0],
7                     [0,  0, 1]])
8
9 theta = np.pi / 4
10 R = rz(theta) # 이 장의 능동적 관례, +theta = CCW
11 p_robot = np.array([1.0, 1.0, 2.0])
12
13 p_global = R @ p_robot # 로봇 좌표계 -> 전역 좌표계 (벡터를 돌린다)
14 p_back = R.T @ p_global # R.T = R^-1 : 전역 -> 로봇 좌표계로 되돌리기
15 # R.T를 쓰는 순간 lec01의 수동적 관례와 같아진다
```

Listing 2.1: R_Z 로 점을 전역/로봇 좌표계 사이에서 옮기기.

장 3 에서 다룬 시뮬레이터 Webots 와 ARGoS 는 모두 차동구동 (differential-drive) 로봇을 전역 좌표계와 로봇 좌표계 두 틀 모두로 조작할 수 있게 해 준다. 시뮬레이터의 API 가 로봇 좌표계 기준 속도 (전진 속도, 각속도) 를 받으면, 화면에 그려지는 전역 좌표계의 위치·방향은 결국 이 절의 R_Z (또는 장 1 의 R_Z^T) 로 매 스텝 갱신된 결과다.

2.5 앞으로

다리 이동 (legged locomotion) 은 바퀴 이동에 비해 아직 연구 단계에 머무는 부분이 많다. 걸음새 최적화 (gait optimization) 와 탄성 에너지 저장을 결합해 효율을 끌어올리는 문제, 속도와 에너지 효율 사이의 절충, 걷는 동안 실시간으로 균형을 잡는 동적 균형 제어 (dynamic balance control), 발을 디딜 위치를 정하기 위한 정밀한 지형 인식 (terrain perception) 이 모두 열린 문제로 남아 있다. 반대로 바퀴 이동에서는 방향과 위치를 다루는 수학이 이미 정리되어 있으므로, 다음으로 이어지는 주제는 바퀴가 지켜야 하는 운동학적 제약, 즉 구름 (rolling) 과 옆으로 미끄러지지 않는다는 (no lateral sliding) 제약이다. 이는 장 1 의 운동 제어 (motion control) 절에서 이어받는다.

Part II

Assignments

Chapter 3

Assignment 1 — Simulator Installation

3.1 Environment

All work was done on a MacBook Pro with an Apple M3 Max processor (`arm64`), running macOS with Homebrew 6.0.21 and CMake 4.4.3. The CMake version matters: it is the source of most of the build failures recorded below.

3.2 Webots

Webots is distributed both as a disk image and as a Homebrew cask (<https://cyberbotics.com/>). Either path works; the cask was used.

```
# option 1 - download the disk image and run it
# https://github.com/cyberbotics/webots/releases/download/R2025a/webots-R2025a.dmg

# option 2 - Homebrew cask
brew install --cask webots
```

Listing 3.1: Installing Webots.

3.3 ARGoS

ARGoS is documented at <https://www.argos-sim.info/> and is built from source at <https://github.com/ilpinczy/argos3>.

3.3.1 The Homebrew tap fails

The maintainer publishes a tap, and the tap owner and the repository owner are the same account, so the tap looked like the safe route.

```
brew tap ilpinczy/argos3
brew trust ilpinczy/argos3
brew install bash-completion qt argos3
```

Listing 3.2: The tap route.

It fails during CMake configuration:

```

> experiments git:(master) x l
total 168
drwxr-xr-x@ 20 tio  staff  640B Sep  3 17:34 .
drwxr-xr-x@ 12 tio  staff  384B Sep  5 18:00 ..
-rw-r--r--@ 1 tio  staff  2.7K Sep  3 17:34 custom_distributions.argos
-rw-r--r--@ 1 tio  staff  6.5K Sep  3 17:34 diffusion_1.argos
-rw-r--r--@ 1 tio  staff  3.6K Sep  3 17:34 diffusion_10.argos
-rw-r--r--@ 1 tio  staff  3.6K Sep  3 17:34 epuck_obstacleavoidance.argos
-rw-r--r--@ 1 tio  staff  2.8K Sep  3 17:34 eyebot_circle.argos
-rw-r--r--@ 1 tio  staff  2.4K Sep  3 17:34 eyebot_flocking.argos
-rw-r--r--@ 1 tio  staff  3.1K Sep  3 17:34 flocking.argos
-rw-r--r--@ 1 tio  staff  5.2K Sep  3 17:34 foraging.argos
-rw-r--r--@ 1 tio  staff  3.3K Sep  3 17:34 galib-trial.argos
-rw-r--r--@ 1 tio  staff  3.0K Sep  3 17:34 galib.argos
-rw-r--r--@ 1 tio  staff  2.0K Sep  3 17:34 gripping.argos
-rw-r--r--@ 1 tio  staff  3.5K Sep  3 17:34 id.argos
-rw-r--r--@ 1 tio  staff  3.8K Sep  3 17:34 manualcontrol.argos
-rw-r--r--@ 1 tio  staff  3.3K Sep  3 17:34 mpga-trial.argos
-rw-r--r--@ 1 tio  staff  3.0K Sep  3 17:34 mpga.argos
-rw-r--r--@ 1 tio  staff  5.0K Sep  3 17:34 multiple_engines.argos
-rw-r--r--@ 1 tio  staff  2.6K Sep  3 17:34 synchronization.argos
-rw-r--r--@ 1 tio  staff  3.9K Sep  3 17:34 trajectory.argos
> experiments git:(master) x pwd
/Users/tio/Documents/temp/argos3-examples/experiments

```

Figure 3.1: Webots installed on macOS.

```

CMake Error at CMakeLists.txt:15 (cmake_policy):
  Compatibility with CMake < 3.5 has been removed from CMake.


Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
to tell CMake that the project requires at least <min> but has been updated
to work with policies introduced by <max> or earlier.


Or, add -DCMAKE_POLICY_VERSION_MINIMUM=3.5 to try configuring anyway.

-- Configuring incomplete, errors occurred!

```

Listing 3.3: CMake rejects the project’s minimum version.

The cause is the CMake version: the local CMake is 4.4.3, and ARGoS still declares compatibility with CMake < 3.5, which CMake 4 no longer accepts. The error message names its own fix — `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` — but the tap gives no place to pass it. So the tap was removed and ARGoS was built by hand instead.

```

cmake --version      # cmake version 4.4.3
brew untap ilpincy/argos3

```

Listing 3.4: Removing the tap.

3.3.2 Building from source

Building by hand makes it possible to pass the policy flag, and `-DCMAKE_INSTALL_PREFIX=$HOME/.local` keeps the install in user space so no `sudo` is needed.

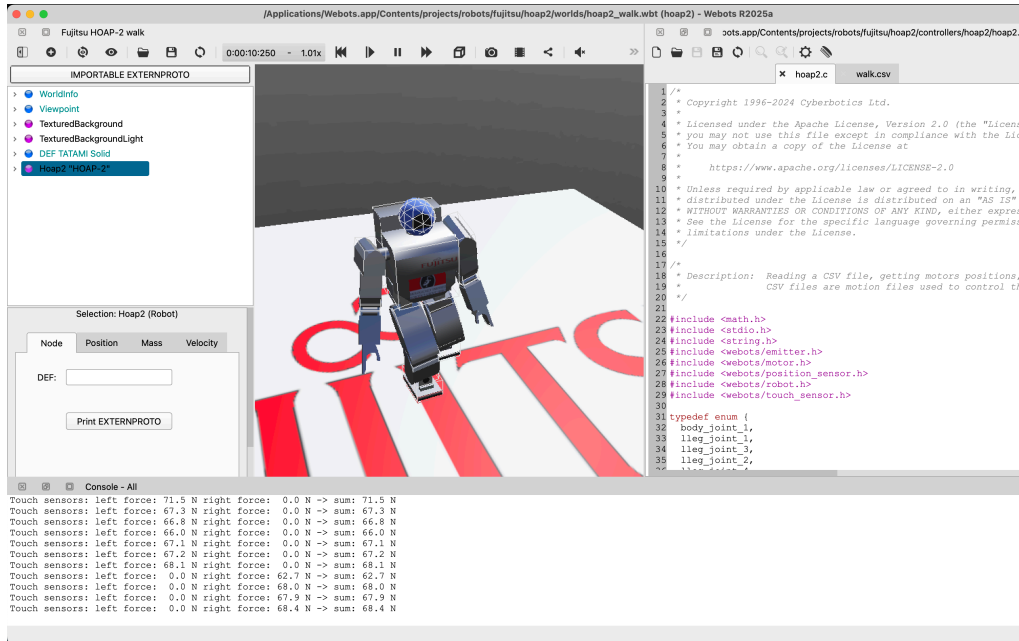


Figure 3.2: Webots running a sample world.

```
brew install pkg-config cmake libpng freeimage lua qt docbook asciidoc \
graphviz doxygen

git clone https://github.com/ilpinczy/argos3.git argos3
cd argos3
mkdir build_simulator
cd build_simulator

cmake ../src -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \
-DCMAKE_INSTALL_PREFIX=$HOME/.local \
-DCMAKE_PREFIX_PATH=$(brew --prefix qt)

make
make doc
make install
```

Listing 3.5: Dependencies and source build.

Two failures were hit on the way to that command, and both are already fixed in it:

1. **make install cannot find argos3.1.gz.** The man page is a build target of its own. Running `make doc` before `make install` produces it.
2. **find_package(ARGoS) cannot find FindARGoSQTOpenGL.cmake** when building the examples. The Qt OpenGL visualization was not built, because CMake did not find Qt. Adding `-DCMAKE_PREFIX_PATH=$(brew --prefix qt)` and rebuilding fixes it.

3.3.3 Verifying the installation

Because the prefix is `$HOME/.local`, its bin directory must be on `PATH`; otherwise `argos3` reports `command not found` even though it installed correctly.

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

argos3 -v
```

Listing 3.6: Putting the binary on `PATH`.

3.3.4 Running an example

The example worlds are listed at <https://www.argos-sim.info/examples.php>.

```
cd argos3-examples
mkdir build
cd build
cmake ../ -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \
        -DCMAKE_PREFIX_PATH=$HOME/.local
make
```

Listing 3.7: Building the examples.

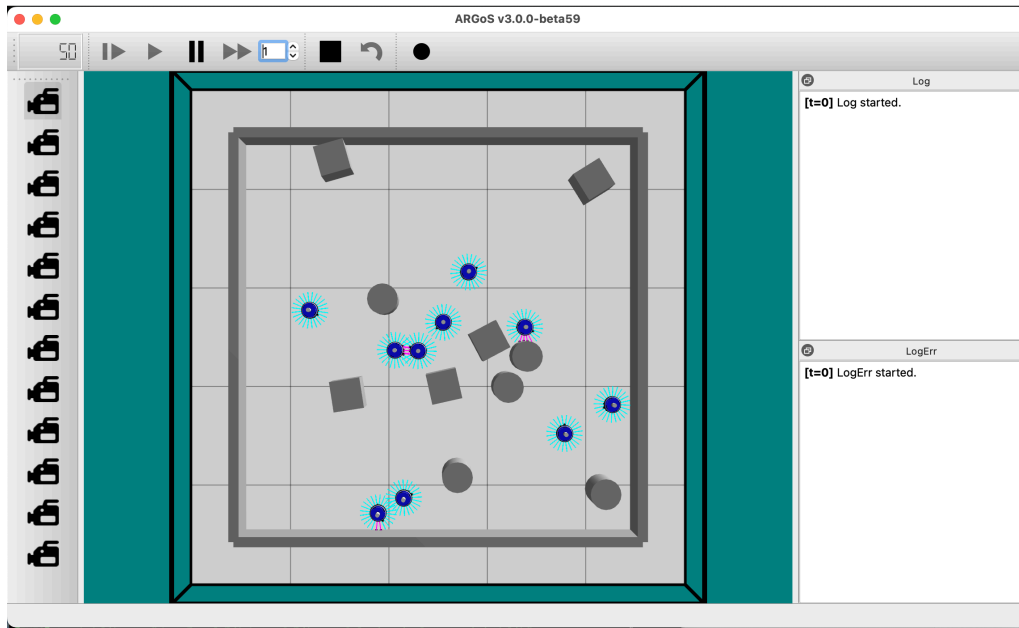


Figure 3.3: ARGoS running an example experiment.

3.3.5 Uninstalling

The install writes its own uninstall script — the install log lists `$HOME/.local/share/argos3/uninstall_argos3.sh`. That is the official removal path and it should be preferred.

```
bash ~/.local/share/argos3/uninstall_argos3.sh

# if the script is missing, remove the installed paths by hand.
# no sudo needed - everything is under the ~/.local prefix.
rm -f ~/.local/bin/argos3
rm -rf ~/.local/include/argos3
rm -rf ~/.local/lib/argos3
rm -f ~/.local/lib/libargos3*
rm -rf ~/.local/share/argos3

which argos3          # no output means the removal is complete
```

Listing 3.8: Uninstalling ARGoS.

Chapter 4

Assignment 2 — Rotation Matrices

4.1 배경 — 회전 행렬은 무엇이고 왜 필요한가

과제는 행렬을 주고 곱하라고만 한다. 이 절은 그 행렬이 어디에서 왔고 왜 필요한지를 정리한다. 이후 절들은 모두 영어로 되어 있다.

4.1.1 왜 필요한가 — 좌표계가 여러 개다

로봇은 좌표계 (frame) 를 하나만 쓰지 않는다. 월드 좌표계, 로봇 베이스 좌표계, 관절마다 하나씩, 카메라 좌표계, 그리퍼 좌표계가 동시에 존재한다. 공간의 같은 한 점이 좌표계마다 다른 세 숫자로 표현된다는 뜻이다. 카메라가 “물체가 내 앞 0.5m” 라고 말할 때, 팔을 움직이려면 그 값을 베이스 좌표계의 값으로 바꿔야 한다.

회전 행렬 R 이 그 변환기다. 좌표계 사이의 방향 (orientation) 차이를 담당한다. 좌표계의 원점들이 서로 떨어져 있는 경우, 즉 위치 차이는 평행이동으로 따로 처리하며, 보통 회전과 평행이동을 하나로 묶은 4×4 동차 변환 행렬 (homogeneous transformation) 을 쓴다. 이번 과제는 그중 회전 부분만 다룬다.

4.1.2 두 가지 해석

같은 행렬을 두 가지로 읽을 수 있고, 둘을 섞으면 부호에서 틀린다.

- 능동적 (active) 해석 — 좌표계는 그대로 두고 벡터를 돌린다. 물체가 실제로 회전한 것이다.
- 수동적 (passive) 해석 — 벡터는 그대로 두고 좌표계를 돌린 뒤 같은 점을 새 좌표계에서 다시 표현한다. 이때 쓰는 행렬은 R^T 로, 능동적 회전의 역이다.

이 장은 능동적 해석을 쓴다. v_1 을 돌려서 v_2 를 얻는다. 각도의 부호는 오른손 법칙을 따른다 — 회전축의 양의 방향에서 원점을 내려다볼 때 반시계 방향이 양수다.

4.1.3 어떻게 만들어지는가 — 2 차원에서의 유도

평면 위의 점 (x, y) 를 원점 기준 극좌표로 쓴다. 반지름을 r , 시작 각도를 α 라 하면

$$x = r \cos \alpha, \quad y = r \sin \alpha. \quad (4.1)$$

여기서 θ 만큼 더 돌리면 반지름은 그대로이고 각도만 $\alpha + \theta$ 가 된다 (그림 4.1). 삼각함수 덧셈정리를 적용하고 식 (4.1) 을 다시 대입하면

$$x' = r \cos(\alpha + \theta) = r \cos \alpha \cos \theta - r \sin \alpha \sin \theta = x \cos \theta - y \sin \theta, \quad (4.2)$$

$$y' = r \sin(\alpha + \theta) = r \sin \alpha \cos \theta + r \cos \alpha \sin \theta = x \sin \theta + y \cos \theta. \quad (4.3)$$

행렬로 묶으면

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (4.4)$$

회전 행렬은 여기서 나온다. 새로 정의한 것이 아니라 덧셈정리를 행렬로 적은 것일 뿐이다.

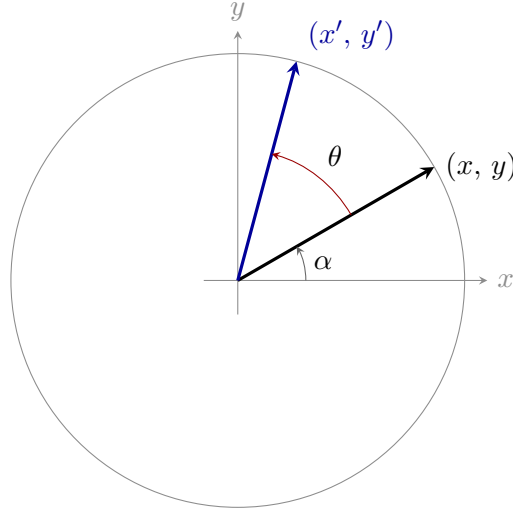


그림 4.1: 2 차원 회전. 반지름 r 은 변하지 않고 각도만 α 에서 $\alpha + \theta$ 로 늘어난다.

4.1.4 3 차원으로 확장하기

어떤 축을 중심으로 돌리면 그 축 방향 성분은 변하지 않는다. 그래서 3 차원 회전 행렬은 식 (4.4) 의 2×2 블록을 나머지 두 좌표 자리에 끼워 넣고, 회전축 자리에는 1 을 놓은 형태가 된다.

Z 축 회전이 가장 직관적이다. z 는 그대로이고 (x, y) 가 식 (4.4) 그대로 돌아간다 — 그 결과가 식 (4.9) 이다. X 축 회전도 같다. x 는 그대로이고 (y, z) 가 돌아간다 (식 (4.7)).

Y 축 회전에서 부호가 뒤집혀 보이는 것이 늘 걸리는 부분이다. 이유는 순환 순서에 있다. 오른손 좌 표계의 순환 순서는 $x \rightarrow y \rightarrow z \rightarrow x$ 이고, Y 축을 뺀 나머지 쌍은 (y, z) 나 (x, y) 가 아니라 (z, x) 다. 즉 식 (4.4) 에서 x 자리에 z 를, y 자리에 x 를 넣어야 한다.

$$\begin{bmatrix} z' \\ x' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} z \\ x \end{bmatrix} \implies \begin{aligned} z' &= z \cos \theta - x \sin \theta, \\ x' &= z \sin \theta + x \cos \theta. \end{aligned} \quad (4.5)$$

이것을 (x, y, z) 순서로 다시 쓰면 $+\sin \theta$ 가 오른쪽 위로, $-\sin \theta$ 가 왼쪽 아래로 가서 식 (4.8) 의 모양이 된다. R_Y 만 부호가 다른 것이 아니라, (z, x) 순서를 (x, z) 로 되돌리면서 자리가 바뀐 것이다.

4.1.5 회전 행렬의 성질

- **열 (column) 은 기저 벡터가 옮겨간 자리다.** R 의 j 번째 열은 단위 벡터 $\hat{e}_j$ 를 돌린 결과다. 식 (4.9) 에서 $R_Z(\theta)$ 의 첫 열 $(\cos \theta, \sin \theta, 0)^T$ 가 바로 $\hat{x}$ 가 도착한 지점이다.
- **직교 행렬이다.** $R^T R = I$ 이므로 $R^{-1} = R^T$ 다. 회전을 되돌리려면 전치만 하면 된다 — 역행렬 을 계산할 필요가 없다.
- **$\det R = +1$ 이다.** 부피와 손방향 (handedness) 을 보존한다. $\det = -1$ 이면 반사가 섞인 것이고, 그것은 회전이 아니다.
- **길이와 각도를 보존한다.** 이것이 강체 (rigid) 회전이라는 말의 뜻이다. 물체는 방향만 바뀌고 모양 은 바뀌지 않는다.

마지막 성질은 답을 계산하는 데 바로 쓸 수 있다. $\|v_1\|^2 = 1 + 4 + 9 = 14$ 이고, 식 (4.11) 와 식 (4.12) 의 정확한 값으로 계산하면

$$\|v_2\|^2 = 3^2 + \left(\frac{3\sqrt{2}}{2}\right)^2 + \left(\frac{\sqrt{2}}{2}\right)^2 = 9 + \frac{9}{2} + \frac{1}{2} = 14, \quad (4.6)$$

$$\|v_3\|^2 = (-2)^2 + (-\sqrt{2})^2 + (2\sqrt{2})^2 = 4 + 2 + 8 = 14.$$

세 값이 모두 같다. 다만 이것은 필요조건이지 충분조건이 아니다 — 길이가 보존됐다고 해서 회전 순서나 각도가 맞다는 보장은 없다. 틀린 답을 걸러내는 값싼 점검이다.

4.1.6 왜 세 번 나눠서 곱하는가

3 차원에서 가능한 모든 방향은 숫자 세 개로 표현된다. 그래서 임의의 회전을 기본 회전 세 개의 곱으로 분해할 수 있고, 그 세 각도를 오일러 각 (Euler angles) 이라 부른다. 실용적인 이유도 있다 — 로봇 관절은 보통 한 축을 중심으로 돈다. 관절 각도가 그대로 기본 회전의 각도가 되므로, 링크마다 기본 회전 행렬을 하나씩 만들어 곱하면 말단 장치의 자세가 나온다. 이것이 정기구학 (forward kinematics) 의 골격이다.

그런데 **회전은 교환법칙이 성립하지 않는다**. 일반적으로 $R_A R_B \neq R_B R_A$ 다. 이 과제의 2 번과 3 번 문제가 정확히 그 시연이다. 두 문제는 같은 벡터에 같은 세 행렬 $R_X(\varphi)$, $R_Y(\theta)$, $R_Z(\psi)$ 를 쓴다. 다른 것은 적용 순서뿐인데, v_2 와 v_3 은 표 4.1 에서 보듯 전혀 다른 벡터다. 그래서 회전을 말할 때는 각도만으로 부족하고 순서를 반드시 함께 명시해야 한다. 곱하는 순서의 규칙은 절 4.2.1 에서 다룬다.

또 한 가지, 여기서 말하는 X , Y , Z 축은 회전과 무관하게 고정된 월드 좌표축이다 (외재적, extrinsic). 물체와 함께 움직이는 자기 축을 기준으로 돌리는 내재적 (intrinsic) 회전으로 해석하면 곱하는 순서가 반대가 된다.

4.2 Elementary rotation matrices

The three elementary rotations about the coordinate axes are

$$R_X(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}, \quad (4.7)$$

$$R_Y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad (4.8)$$

$$R_Z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (4.9)$$

4.2.1 Order of composition

A composite rotation is built by *applying* the elementary rotations one after another. If the vector is first rotated about X by φ , then about Y by θ , and finally about Z by ψ , each matrix multiplies the result produced by the previous one:

$$v_2 = R_Z(\psi) (R_Y(\theta) (R_X(\varphi) v_1)) = \underbrace{R_Z(\psi) R_Y(\theta) R_X(\varphi)}_{R_{XYZ}} v_1. \quad (4.10)$$

The matrix applied *first* therefore stands *rightmost*, next to the vector. This is the convention used throughout this chapter and in the code; writing the product in the reading order $R_X R_Y R_Z$ would describe a different rotation.

4.3 Implementation

The provided skeleton `HW2_Rotation_matrices_skeleton.py` was completed by adding R_Y and R_Z (Equations (4.8) and (4.9)), the three angle values, and the two composite products. The complete program is listed below; it is the same file submitted for the assignment.

```

1 import numpy as np
2 import math as m
3
4 # rotation matrix on x axis
5 def Rx(theta):
6     return np.matrix([[ 1, 0, 0],
7                        [ 0, m.cos(theta), -m.sin(theta)],
8                        [ 0, m.sin(theta), m.cos(theta)]])
9
10 # rotation matrix on y axis
11 def Ry(theta):
12     return np.matrix([[ m.cos(theta), 0, m.sin(theta)],
13                        [ 0, 1, 0],
14                        [-m.sin(theta), 0, m.cos(theta)]])
15
16 # rotation matrix on z axis
17 def Rz(theta):
18     return np.matrix([[ m.cos(theta), -m.sin(theta), 0],
19                        [ m.sin(theta), m.cos(theta), 0],
20                        [ 0, 0, 1]])
21
22
23 # please give the values of the three angles
24 phi = m.pi/2
25 theta = m.pi/4
26 psi = m.pi/2
27 print("phi =", phi)
28 print("theta =", theta)
29 print("psi =", psi)
30
31
32 # given the vector v1 = (1,2,3), calculate the vector v2 after the XYZ rotation.
33 # X first, then Y, then Z: the matrix applied last stands leftmost.
34 v1 = np.array([[1],[2],[3]])
35 v2 = Rz(psi) * Ry(theta) * Rx(phi) * v1
36 print("The vector v2 is", np.round(v2, decimals=2))
37
38 # given the vector v1 = (1,2,3), calculate the vector v3 after the YZX rotation.
39 v3 = Rx(phi) * Rz(psi) * Ry(theta) * v1
40 print("The vector v3 is", np.round(v3, decimals=2))

```

Listing 4.1: Lim_20701313_hw2.py — complete solution.

4.4 Question 2 — XYZ rotation

Given $\varphi = \pi/2$, $\theta = \pi/4$, $\psi = \pi/2$ and $v_1 = (1, 2, 3)^T$. Evaluating Equations (4.7) to (4.9) at these angles ($\cos \frac{\pi}{2} = 0$, $\sin \frac{\pi}{2} = 1$, $\cos \frac{\pi}{4} = \sin \frac{\pi}{4} = \frac{\sqrt{2}}{2}$):

$$R_X\left(\frac{\pi}{2}\right) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad R_Y\left(\frac{\pi}{4}\right) = \begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}, \quad R_Z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Applying them in the order of Equation (4.10), one step at a time:

$$\begin{aligned}
a &= R_X\left(\frac{\pi}{2}\right) v_1 = \begin{bmatrix} 1 \\ -3 \\ 2 \end{bmatrix}, \\
b &= R_Y\left(\frac{\pi}{4}\right) a = \begin{bmatrix} \frac{\sqrt{2}}{2}(1+2) \\ -3 \\ \frac{\sqrt{2}}{2}(2-1) \end{bmatrix} = \begin{bmatrix} \frac{3\sqrt{2}}{2} \\ -3 \\ \frac{\sqrt{2}}{2} \end{bmatrix}, \\
v_2 &= R_Z\left(\frac{\pi}{2}\right) b = \begin{bmatrix} 3 \\ \frac{3\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}. \\
v_2 &= \left(3, \frac{3\sqrt{2}}{2}, \frac{\sqrt{2}}{2}\right)^\top \approx (3.00, 2.12, 0.71)^\top.
\end{aligned} \tag{4.11}$$

4.5 Question 3 — YZX rotation

The same angles and the same v_1 , but now the rotation about Y is applied first, then Z , then X , so the composite matrix is $R_X(\varphi) R_Z(\psi) R_Y(\theta)$:

$$\begin{aligned}
c &= R_Y\left(\frac{\pi}{4}\right) v_1 = \begin{bmatrix} \frac{\sqrt{2}}{2}(1+3) \\ 2 \\ \frac{\sqrt{2}}{2}(3-1) \end{bmatrix} = \begin{bmatrix} 2\sqrt{2} \\ 2 \\ \sqrt{2} \end{bmatrix}, \\
d &= R_Z\left(\frac{\pi}{2}\right) c = \begin{bmatrix} -2 \\ 2\sqrt{2} \\ \sqrt{2} \end{bmatrix}, \\
v_3 &= R_X\left(\frac{\pi}{2}\right) d = \begin{bmatrix} -2 \\ -\sqrt{2} \\ 2\sqrt{2} \end{bmatrix}. \\
v_3 &= \left(-2, -\sqrt{2}, 2\sqrt{2}\right)^\top \approx (-2.00, -1.41, 2.83)^\top.
\end{aligned} \tag{4.12}$$

4.6 Results

Table 4.1 collects the two answers; both agree with the hand computation in Equations (4.11) and (4.12).

Table 4.1: Rotated vectors for $\varphi = \pi/2$, $\theta = \pi/4$, $\psi = \pi/2$, $v_1 = (1, 2, 3)^\top$.

Vector	Order	Exact	Rounded (2 d.p.)
v_2	XYZ	$\left(3, \frac{3\sqrt{2}}{2}, \frac{\sqrt{2}}{2}\right)$	$(3.00, 2.12, 0.71)$
v_3	YZX	$\left(-2, -\sqrt{2}, 2\sqrt{2}\right)$	$(-2.00, -1.41, 2.83)$

The program output is reproduced verbatim:

```
phi = 1.5707963267948966
theta = 0.7853981633974483
psi = 1.5707963267948966
The vector v2 is [[3.  ]
 [2.12]
 [0.71]]
The vector v3 is [[-2.  ]
 [-1.41]
 [ 2.83]]
```

Listing 4.2: Console output of the program.

Chapter 5

Assignment 3 — Low-Pass Filtering of an Image

5.1 The problem

A low-pass filter smooths an image. The provided image is smoothed with three box filters of size 3×3 , 5×5 and 7×7 , implemented in the `conv` function of the provided template `Low_pass_filter_template.py`. New code was added, but the template itself was not changed.

A box filter replaces the value at the centre of the box with the average of the values inside it. For the 3×3 case the filter matrix is

$$filter = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}. \quad (5.1)$$

The assignment's worked example takes the sub-image

$$subImage_1 = \begin{bmatrix} 21 & 78 & 63 \\ 16 & 109 & 73 \\ 6 & 88 & 86 \end{bmatrix}, \quad (5.2)$$

and multiplies it element-wise by the filter,

$$C = filter * subImage_1 = \begin{bmatrix} 2.33 & 8.67 & 7 \\ 1.78 & 12.11 & 8.11 \\ 0.67 & 9.78 & 9.56 \end{bmatrix}. \quad (5.3)$$

Summing the entries of C should reproduce the mean of $subImage_1$, because that is exactly what a box filter is defined to compute. The check is done one operation at a time in Equation (5.4).

$2.33 + 8.67 + 7 = 18$	row 1 of C
$1.78 + 12.11 + 8.11 = 22$	row 2 of C
$0.67 + 9.78 + 9.56 = 20$	row 3 of C
$18 + 22 + 20 = 60$	sum of C

$21 + 78 + 63 + 16 + 109 + 73 + 6 + 88 + 86 = 540$	sum of $subImage_1$	(5.4)
$540/9 = 60$	mean of $subImage_1$	

The two routes agree at 60, confirming that the filter matrix does perform the averaging. Running the finished `conv` function (Section 5.2) on `subImage1` with the 3×3 filter returns exactly 60.0, matching this hand calculation. The 5×5 and 7×7 filters are built the same way, with $1/25$ and $1/49$ in every entry.

5.2 Implementation

The template left two gaps: `lowPassFilter`, which builds the filter matrix, and `conv`, which slides that filter over the image.

Problem: the filter entries must sum to 1. If the filter’s entries summed to anything other than 1, every output pixel would be a value scaled away from the true local average, and the whole image would come out uniformly brighter or darker regardless of what it originally showed. An averaging filter must leave the image’s overall brightness unchanged; only the high-frequency detail (fine texture, grain) should be removed. `lowPassFilter` solves this by filling an $lpfw \times lpfh$ matrix with the single value $1/(lpfw \cdot lpfh)$ in every entry:

$$\text{lowPass} = \text{np.ones}((lpfw, lpfh)) / (lpfw * lpfh). \quad (5.5)$$

Every entry is $1/(lpfw \cdot lpfh)$ and there are $lpfw \cdot lpfh$ of them, so the entries sum to exactly 1 by construction, for any box size.

Problem: what happens at the boundary of the image. A filter of size $fw \times fh$ needs a full $fw \times fh$ neighbourhood centred at each output pixel. Near the edge of the image that neighbourhood runs off the array, so there is no well-defined average to compute there. `conv` solves this by simply not producing an output for those positions: the sliding window only visits positions where the full window still fits inside the input, which is why the output array is declared smaller than the input,

$$\text{out.shape} = (iw - fw + 1, ih - fh + 1), \quad (5.6)$$

and the double loop in `conv` fills each position with the element-wise sum of the filter against the matching sub-image:

$$\text{out}[i, j] = \text{np.sum}(\text{image}[i:i+fw, j:j+fh] * \text{filter}). \quad (5.7)$$

For the 512×512 input this drops the boundary pixels, giving 510×510 , 508×508 and 506×506 outputs for the 3×3 , 5×5 and 7×7 filters respectively (Table 5.1).

The only other addition beyond `lowPassFilter` and `conv` is a guard placed right after the image is read,

$$\text{if img.ndim} == 3: \text{img} = \text{img.mean}(\text{axis}=2), \quad (5.8)$$

because the template’s `iw, ih = image.shape` assumes a single-channel image and would fail on a 3-channel one. This is the one place where the template’s assumption is fragile: for this input the guard did not actually trigger, since `original.jpg` is a baseline single-component JPEG and `mpimg.imread` already returned a $(512, 512)$ uint8 array, but the guard protects against inputs that are not.

No normalization step was needed either. The image is a uint8 array and the filter is a float64 array, so their product upcasts to float64; because the filter entries sum to 1 (Equation (5.5)), the averaged values never leave the range 0 to 255, and `cv2.imwrite` saturate-casts the result back to 8 bits when writing the file (it prints a benign “Unsupported depth ...fallbacked to CV_8U” warning in the process, shown in Listing 5.2).

The complete program is listed below; it is the same file submitted for the assignment.

```

1 import cv2
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import math
5 import matplotlib.image as mpimg
6 import pdb
7 # imshow: Function that handles both rgb and grayscale image and shows them
   accordingly.
8 # Parameters:
9 #   img: input img
10 # Return: None
11
12 def imshow(img):
13     if len(img.shape) < 3:
14         plt.imshow(img, cmap="gray")
15     elif img.shape[2] == 1:
16         plt.imshow(np.resize(img, (img.shape[0], img.shape[1])), cmap="gray")
17     else:
18         plt.imshow(img)
19
20 # low pass filter
21 def lowPassFilter(lpfw, lpfh):
22     print("Please create a (lpfw x lpfh) Low Pass Filter matrix")
23
24     #***** please add your code here *****
25     lowPass = np.ones((lpfw, lpfh)) / (lpfw * lpfh)
26
27     print(lowPass)
28     return lowPass
29
30 # it applies the filter on the image and generates a new image
31 # input: image, filter
32 # output: out
33 def conv(image, filter):
34     iw, ih = image.shape
35     fw, fh = filter.shape
36     out = np.zeros((iw-fw+1, ih-fh+1)) #it does not include the pixel on the
        boundary
37
38     # please select fw x fh sub images in the image. Then, apply the filter on the
        sub images.
39
40     #***** please add your code here *****
41     for i in range(iw - fw + 1):
42         for j in range(ih - fh + 1):
43             subImage = image[i:i+fw, j:j+fh]
44             out[i, j] = np.sum(subImage * filter)
45
46     # Return the resulting image "out"
47     return out
48
49 img = mpimg.imread("original.jpg")
50 # original.jpg (512x512) is already single-channel, but mpimg.imread can
51 # return a 3-channel array for other images, which breaks the template's
52 # `iw, ih = image.shape` in conv(). Guard by averaging channels to grayscale.
53 if img.ndim == 3:
54     img = img.mean(axis=2)
55 imshow(img)
56 plt.show()
57 print(img.shape)
58
59

```

```

60 fig = plt.figure(1, figsize=(18, 16))
61 print(" ===== Low Pass Filter =====")
62
63 for i in range(3,8,2):
64     # subplot the figure in 1x3
65     splot = plt.subplot(1, 3, (i-1)//2)
66     splot.set_xlabel(str(i)+"x"+str(i))
67
68     # use different box size to filter the image
69     (lpfw,lpfh) = (i,i)
70     low_image = conv(img, lowPassFilter(lpfw, lpfh))
71     imshow(low_image)
72     cv2.imwrite("low"+str(i)+"x"+str(i)+".jpg",low_image)
73 plt.show()

```

Listing 5.1: Lim_20701313_hw3.py: the complete solution.

5.3 Results

Figure 5.1 shows the original image next to the three filtered outputs. Table 5.1 lists the corresponding filter entry value and output shape for each box size.

Table 5.1: Filter size, entry value and output shape for each run.

Box size	Filter entry value	Output shape
3×3	$1/9 \approx 0.11111111$	510×510
5×5	$1/25 = 0.04$	508×508
7×7	$1/49 \approx 0.02040816$	506×506

The console output of the run is reproduced verbatim, with the 5×5 and 7×7 filter matrices abbreviated since every entry is identical:

```

(512, 512)
===== Low Pass Filter =====
Please create a (lpfw x lpfh) Low Pass Filter matrix
[[0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]]
Please create a (lpfw x lpfh) Low Pass Filter matrix
[[0.04 0.04 ... ]] # 5x5, every entry 0.04
Please create a (lpfw x lpfh) Low Pass Filter matrix
[[0.02040816 ... ]] # 7x7, every entry 0.02040816

```

Listing 5.2: Console output of the program.

The smoothing grows with the box size, exactly as expected from averaging over more neighbours per output pixel. At 3×3 the film grain is only lightly softened and fine detail such as hair strands, eyelashes and skin texture stays sharp. At 5×5 the grain is largely gone and fine hair strands start merging, while the eyes, nose and lips stay well defined. At 7×7 the smoothing is strongest: the background and skin flatten, hair strands merge into soft masses, and edges such as the jawline show visible softening.



Original (512×512).



3×3 box filter (510×510).



5×5 box filter (508×508).



7×7 box filter (506×506).

Figure 5.1: Original image and low-pass filtered outputs at three box sizes.